

Documentacion tecnica - Delivery 2

Secciones nuevas y actualizacion de lo que cambio respecto a Delivery 1. El analisis toma como base el commit **dc52d6c** y compara hasta **17ff1ea** en main.

Dato	Valor
Proyecto	VibeBuilder - Android app para crear web apps con prompts de IA.
Base E1	Commit dc52d6c - flujo de detalle, preview, API y Gradle 8.13.
Cierre E2 analizado	Commit 17ff1ea - regeneracion robusta de versiones.
Archivo solicitado	doc_tecnica_e2.pdf
Fecha	23/06/2026

Resumen ejecutivo

- Delivery 2 transforma el prototipo de E1 en un builder mas controlable: proyectos editables, historial mas rico, preview por version y regeneracion de fallos.
- El backend deja de depender solo de un HTML/preview temporal y guarda artefactos versionados con manifiesto, archivos, checksums y exportacion ZIP.
- La app Android incorpora mejor manejo de estados, diseno visual unificado, configuracion de v0 y acciones de recuperacion.
- La autenticacion completa con usuarios y roles todavia no esta implementada; el alcance actual usa sesion por dispositivo y aislamiento por owner session.

1. Cambios principales desde Delivery 1

- Integracion con v0 para generar aplicaciones reales y recuperar archivos generados cuando el proveedor los expone.
- Backend preparado para despliegue en Vercel, con base configurable desde Android y persistencia opcional en Turso mediante libSQL.
- Nuevo modelo de artefactos: `version_artifacts` y `version_artifact_files` separan metadata, archivos, checksums y storage.
- Preview mejorado: se puede resolver el preview actual o uno historico con `target=current` o `target=version`.
- Historial de versiones enriquecido con estados `queued`, `generating`, `validating`, `success`, `failed` y `cancelled`.
- Regeneracion robusta: una version fallida puede regenerarse sin borrar historial ni cambiar el current version hasta que el intento sea exitoso.
- Gestion de proyectos: edicion, eliminacion logica, busqueda y ordenamiento en Home.
- Diseno Compose refinado: componentes reutilizables, tarjetas, controles, spacing, shapes, estados vacios/error/loading.
- Mejoras de pruebas: cobertura backend de proyectos, prompts, preview, artefactos, integracion v0, idempotencia y regeneracion; tests Android de repositorios y ViewModels.

Commits relevantes de E2

Commit	Aporte
87bf640	Integracion v0, preview QR y chat unificado.
7e319f6 - 16dc467	Backend preparado para Vercel y <code>API_BASE_URL</code> configurable.
a5bfe84	Persistencia en Turso cuando <code>TURSO_DATABASE_URL</code> esta configurada.
81ec427	Timeouts largos y recuperacion tras prompts de v0.
3455cd8	Refinamiento del sistema visual mobile en Compose.
9c93bd1	Edicion, eliminacion, busqueda y ordenamiento de proyectos.
4dc4e29	Pipeline de artefactos generados y validacion estructural.
17ff1ea	Regeneracion robusta de versiones fallidas.

2. Autenticacion y autorizacion

Estado implementado

- No se implemento login final con Firebase Auth, JWT propio ni cuentas de usuario completas en Delivery 2.
- El backend exige X-Session-Id con formato UUID v4 para las rutas de proyecto, preview, historial, telemetry e integraciones.
- La app Android obtiene y reutiliza ese identificador mediante SharedPreferencesSessionIdProvider.
- Las operaciones de escritura POST, PUT y PATCH agregan X-Idempotency-Key para evitar duplicados por reintentos de red.
- La key de v0 puede guardarse por sesion si el backend tiene V0_KEYSTORE_SECRET; se devuelve solo estado e indicio enmascarado, nunca la key completa.

Roles y permisos

- Rol real actual: owner de la sesion del dispositivo. No hay administrador, colaborador ni usuario publico.
- Permiso de lectura/escritura: el backend filtra por session_id y deleted_at IS NULL.
- Una sesion no puede actualizar, borrar, listar versiones ni previsualizar proyectos de otra sesion si no coinciden project_id y session_id.
- Las credenciales v0 se administran en /integrations/v0 y pertenecen a la misma sesion.

Proteccion de rutas y pantallas

Superficie	Proteccion actual
Android	Navegacion local sin roles. Project Detail, Preview, History y v0 Settings consumen API con session id.
Backend proyectos	GET/POST/PATCH/DELETE /projects valida X-Session-Id; PATCH/DELETE exigen pertenencia por session_id.
Backend versiones	Versiones, preview, export y regenerate consultan proyecto por session_id antes de responder.
Integracion v0	GET/PUT/DELETE/POST test en /integrations/v0 opera solo sobre la key guardada de la sesion.

Pendiente para E3

- Agregar autenticacion real con usuarios y token verificable.
- Definir roles para owner, lector publico/forker y eventualmente administrador.
- Proteger la biblioteca comunitaria y el fork con ownership persistente, atribucion y visibilidad.
- Migrar desde identidad de dispositivo hacia identidad de usuario sin perder proyectos locales existentes.

3. Integracion con backend y servicios externos

Endpoints REST usados en Delivery 2

Metodo y ruta	Uso	Request/headers	Response principal
POST /projects	Crear proyecto.	X-Session-Id, X-Idempotency-Key; title, description.	projectId.
GET /projects	Listar proyectos activos.	X-Session-Id.	Lista con id, title, description, currentVersionId, fechas.
PATCH /projects/:id	Renombrar o editar descripcion.	X-Session-Id; title y/o description.	Proyecto actualizado.
DELETE /projects/:id	Eliminar proyecto.	X-Session-Id.	204; soft delete con deleted_at.
POST /projects/:id/prompts	Enviar prompt inicial o iteracion.	X-Session-Id, X-Idempotency-Key; prompt.	promptMessageId, projectVersionId, versionNumber, status, providerMeta.
GET /projects/:id/messages	Cargar chat/historial de prompts.	X-Session-Id.	Mensajes con role, content, versionNumber.
GET /projects/:id/versions	Listar historial de versiones.	X-Session-Id.	Versiones con estado, failureCode, sourceVersionId y artifact opcional.
GET /projects/:id/versions/:versionNumber	Detalle de version.	X-Session-Id.	Detalle con artifact y paths de archivos sin contenido.
GET /projects/:id/versions/:versionNumber/export	Exportar artefacto.	X-Session-Id.	ZIP del proyecto generado.
POST /projects/:id/versions/:versionId/regenerate	Regenerar version fallida.	X-Session-Id, X-Idempotency-Key; prompt opcional.	Nueva version, sourceVersionId y attemptNumber.
GET /projects/:id/preview	Resolver preview actual o historico.	target=current version, versionNumber si aplica.	previewUrl, versionId, versionNumber.
GET /integrations/v0	Estado de conexion v0.	X-Session-Id.	keyStorageAvailable, sessionKeyConfigured, keyHint, envKeyActive.
PUT /integrations/v0	Guardar key v0 por sesion.	apiKey.	204.
DELETE /integrations/v0	Eliminar key v0.	X-Session-Id.	204.
POST /integrations/v0/test	Probar key v0.	apiKey opcional.	ok=true o error V0_CONNECTION_FAILED.
GET /telemetry/summary	Resumen tecnico de eventos.	X-Session-Id.	Totales y eventos recientes.

Persistencia y servicios externos

Servicio/componente	Decision de Delivery 2
SQLite local	Base por defecto para desarrollo y tests. Se aplican migraciones idempotentes desde schema.js.
Turso/libSQL	Persistencia remota opcional cuando TURSO_DATABASE_URL esta configurada.
Vercel	Backend preparado con server.js, http-app.js y vercel.json; Android puede apuntar a API_BASE_URL remoto.
v0	Proveedor externo para generacion. El backend puede usar key global o key guardada por sesion.
Vercel Blob/local storage	Storage de archivos generados. Vercel Blob se usa con BLOB_READ_WRITE_TOKEN; local para desarrollo/tests.
QR/preview externo	Android muestra preview embebido y alternativa de abrir navegador externo o compartir por QR segun la pantalla.

Modelo de datos actualizado

- projects agrega deleted_at para eliminacion logica y conserva current_version_id.
- project_versions agrega provider_meta, preview_url, source_version_id, attempt_number, failure_code, started_at y completed_at.
- prompt_messages conserva el hilo de instrucciones del usuario y respuestas asociadas a version.
- idempotency_requests registra payload_hash y respuesta para evitar duplicados en POST/PUT/PATCH criticos.
- telemetry_events registra create, generate, fail, preview e iterate con metadata sanitizada.
- session_v0_keys guarda ciphertext y key_hint de la key v0 por sesion.
- version_artifacts y version_artifact_files guardan manifiesto, framework, entry point, validation_status, checksums, sizes y storage_key.

Manejo de errores y retry

- Errores estables: SESSION_REQUIRED, IDEMPOTENCY_KEY_REQUIRED, PROJECT_NOT_FOUND, VERSION_NOT_REGENERABLE, PREVIEW_NOT_READY, PREVIEW_EXPIRED, PROVIDER_TIMEOUT, PROVIDER_UNAUTHORIZED, PROVIDER_RATE_LIMITED, entre otros.
- Android traduce timeouts y errores 500 largos en mensajes accionables para revisar Historial o reintentar.
- La regeneracion solo aplica sobre versiones failed y crea una version nueva; no sobrescribe la fallida.
- Los timeouts de prompt suben a 300 segundos porque v0 puede tardar varios minutos.

4. Validacion de salida generada

Implementado

- Se normalizan rutas relativas para bloquear rutas absolutas, null bytes y path traversal.
- Se filtran binarios no soportados y se controlan limites de cantidad de archivos, bytes por archivo y bytes totales.
- Se detecta perfil React + Vite + TS, Next o v0-web a partir de package.json y paths.
- Se parsea package.json para dependency_manifest y se guarda el resultado asociado al artefacto.
- Cada archivo persiste con checksum_sha256, size_bytes, content_type y storage_key.
- La exportacion ZIP se arma desde el artefacto persistido, no desde una respuesta temporal del proveedor.

Limitacion actual

- La validacion actual es estructural. No hay todavia pipeline completo de npm install, TypeScript check, build productivo y health check aislado.
- Por eso validation_status usa structural_ok o structural_failed.
- La deuda tecnica queda registrada para cerrar la validacion completa en una fase siguiente.

Contrato de artefacto

Campo	Proposito
framework	Tipo de proyecto generado: react-vite-ts, next-ts o v0-web.
template_version	Version interna del contrato de artefacto.
entry_point	Archivo de entrada usado para reconstruir o validar.
dependency_manifest	Dependencias extraidas desde package.json.
validation_report	Resultado estructurado de la validacion.
preview_ref	Referencia externa al preview si existe.
provider_chat_id / provider_version_id	Relacion con v0 sin filtrar secretos al cliente.
file_count / total_bytes	Controles de tamano para seguridad y UX.

5. Decisiones de refactoring

Que cambio y por que

- Se reemplazo backend/src/app.js por http-app.js, bootstrap.js y server.js para separar app HTTP, arranque local y despliegue Vercel.
- Se agrego backend/src/artifacts/ para aislar contrato, validacion, extraccion v0, storage y exportacion ZIP.
- Se extrajo database-connection.js para resolver SQLite local o Turso/libSQL segun variables de entorno.
- Se separo generation-provider.js para encapsular proveedor v0, mock provider, timeouts y errores propios.
- Se agrego session-v0-key-store.js para no mezclar manejo de credenciales con rutas HTTP.
- En Android se separaron VibeBuilderApi, RemoteProjectRepository, MockProjectRepository y ServiceLocator para cambiar entre backend real y mock.
- La UI Compose adopto componentes reutilizables AppCard, AppControls, StateViews y tokens de theme/spacing/shapes.

Deuda tecnica identificada

- Completar autenticacion real y roles antes de publicar biblioteca comunitaria o forks entre usuarios.
- Implementar validacion aislada de build para cerrar el ciclo de calidad de artefactos.
- Agregar paginacion/cursor real al historial si los proyectos acumulan muchas versiones.
- Convertir la arquitectura Supabase documentada en implementacion productiva si Delivery 3 necesita apps con datos reales.
- Agregar tests instrumentados de WebView y flujos end-to-end en emulador.
- Revisar comentarios heredados que todavia mencionan previewHtml de Delivery 1 para evitar confusion futura.

6. Evidencia de pruebas

Capa	Archivos/pruebas agregadas o ampliadas
Backend proyectos	projects.post, projects.get, projects.patch-delete, project-detail.
Backend prompts/v0	prompts.post, prompts.post.v0, integrations-v0, v0-provider.
Backend artefactos	artifacts.service, manifest-validator, prompt-integration, extractor, export.
Backend preview/regeneracion	preview.get, versions.regenerate.
Backend seguridad operativa	session-header, session-v0-key-store, telemetry.
Android	RemoteProjectRepositoryTest, ProjectDetailViewModelTest, ProjectListViewModelTest.

7. Conclusion para Delivery 2

Delivery 2 mejora el producto en control, recuperacion y calidad: el usuario puede organizar proyectos, ver historial, previsualizar versiones, configurar v0 y regenerar fallos sin perder el ultimo estado exitoso. La base tecnica queda preparada para Delivery 3 con ownership por sesion, idempotencia, artefactos, storage, telemetry y despliegue remoto.